Designed but Not Used? A Feature Adoption Analysis of LiaScript Courses

Sebastian Zug [1], André Dietrich [1], Ines Aubel [2] und Volker Göhler [1]

Abstract: LiaScript is a Markdown-based language for creating interactive Open Educational Resources that runs entirely client-side: courses are interpreted directly in the learner's browser without any server infrastructure. This architectural decision ensures privacy and decentralisation, but leaves the developers with no telemetry, no usage statistics, and no systematic insight into how the tool's pedagogical affordances are actually employed in practice.

This paper addresses that gap. Using a corpus of 3.317 validated LiaScript courses collected from public GitHub repositories in March 2026, we conduct the first systematic analysis of feature adoption across 307 committers. We introduce a didactically motivated feature taxonomy covering four categories — presentation, interaction, reuse, and embedding — and map it onto the corpus. The analysis reveals a pronounced adoption gradient: while presentation and reuse features are widely used, interactive and embedding affordances remain selectively adopted. A three-group comparison — core developers (462 courses), a prolific school-focused initiative (1 147 courses), and the broader community (1 708 courses) — uncovers sharply distinct usage profiles that would be invisible in aggregate statistics. Notably, prolific community authors exhibit the same feature profile as occasional contributors, indicating that production volume alone does not drive feature exploration. Based on these findings, we identify three adoption gap classes — documentation deficits, workflow barriers, and configuration lock-in — and derive concrete priorities for the further development of LiaScript.

Keywords: Open Educational Resources, LiaScript, Feature Adoption, OER Authoring, Client-side Learning

1 Introduction

The creation of high-quality Open Educational Resources remains a persistent challenge in higher education. While content management systems and dedicated authoring tools exist, they often impose significant technical barriers or proprietary lock-in, discouraging educators from investing in interactive, reusable materials [HO18].

1 TU Bergakademie Freiberg, Institut für Informatik, 09599 Freiberg, Germany, sebastian.zug@informatik.tu-freiberg.de, https://orcid.org/0000-0001-9949-6963; andre.dietrich@informatik.tu-freiberg.de, https://orcid.org/0000-0001-8894-1110; volker.goehler@informatik.tu-freiberg.de, https://orcid.org/0000-0003-1901-1335

2 TU Bergakademie Freiberg, Institut für Technische Chemie, 09599 Freiberg, Germany, ines.aubel@chemie.tu-freiberg.de, https://orcid.org/0000-0002-7589-9634

1.1 The LiaScript Approach

LiaScript [Di19] takes a different approach: it extends standard Markdown with interactive and multimedia elements using a human-readable syntax. Listing 1 illustrates this principle: a single-choice quiz is expressed as a simple list with bracketed markers, directly within the Markdown source.

```
Where does DELFI 2026 take place?

- [( )] Munich
- [(X)] Potsdam
- [( )] Hamburg
[[?]] It is the capital of Brandenburg.
***
Potsdam hosts DELFI 2026 together
with HDI at the University of Potsdam.
***
```

Abb. 1: A LiaScript single-choice quiz: Markdown source (left) and rendered output (right).

Beyond quizzes, the language supports text-to-speech narration, animations, executable code environments, reusable macro libraries, and many more didactic elements — all authored as plain text. Unlike H5P, which requires a server and a Moodle/WordPress installation, or Jupyter Book, LiaScript executes without backend infrastructure. A course file is a standard `.md` file; the LiaScript interpreter is loaded client-side via a single URL. An author pushes the file to any public Git host and shares a link of the form `https://liascript.github.io/course/?URL`. The browser fetches the file and interprets it locally; no data is sent to any LiaScript server.

This client-side architecture is a deliberate design choice with significant advantages: courses can be hosted anywhere, require no accounts, respect learner privacy, and remain accessible even offline. However, it introduces a fundamental blind spot for the developers. Because no requests pass through a central server, *there is no telemetry*. The developers cannot observe which features are used, which courses are accessed, or whether the pedagogical affordances built into the language are exploited in practice.

1.2 Contribution and Research Questions

The only observable proxy for actual usage is the set of publicly available LiaScript courses on GitHub. This paper exploits that proxy systematically. We analyse a corpus of 3.317 validated courses from 307 committers, introducing a didactically motivated taxonomy of LiaScript features and mapping actual adoption rates onto the designed feature space. The result is the first empirical picture of how LiaScript is used in the wild.

A key methodological choice is the segmentation of the corpus into three groups: the LiaScript core developers (Internal), a prolific school-focused power user (MINT-the-GAP),

and the broader community. This separation prevents the highly atypical output of the first two groups from distorting aggregate adoption statistics.

Concretely, we address three research questions:

RQ 1 Which LiaScript features are adopted by the community and which remain underutilised?

RQ 2 How do Internal, MINT-the-GAP, and Community groups differ in their feature adoption profiles?

RQ 3 What development and documentation priorities follow from the identified adoption gaps?

The remainder of the paper is structured as follows. Section 2 reviews related work on OER tools, feature adoption, and repository mining. Section 3 describes the feature taxonomy, the corpus, and the three-group segmentation. Section 4 presents the empirical results. Section 5 derives development priorities from the identified adoption gaps, and Section 6 summarises the contributions.

2 Related Work

Analysing feature adoption in a client-side OER tool requires drawing on three areas of prior work: the *OER authoring tool* landscape, which defines the design space LiaScript operates in; *feature adoption research* in educational technologies, which establishes that gaps between designed and used features are the norm rather than the exception; and *repository mining*, which provides the methodological foundation for studying a tool that produces no server-side telemetry.

2.1 OER Authoring Tools

Several tools address the challenge of creating interactive educational content from plain-text sources. H5P [Fo25; Ki22] is widely adopted for creating self-contained interactive elements (quizzes, interactive videos) that can be embedded in any LMS, but requires a server and does not support version control. Jupyter Book [Be25; Ju25] and mdBook generate complete websites from Markdown files and integrate naturally with Git workflows, but are primarily oriented towards computational and technical content. The *You Only Write Thrice* approach [21] demonstrates that a single Markdown source can generate a book, a slide deck, and a computational notebook simultaneously — a principle LiaScript generalises to the full OER lifecycle. Pandoc-based workflows [KW17] address format interoperability but do not add interactive pedagogical elements. Git-based OER management has also been explored with GitLab as infrastructure [Mo25] and with GitHub specifically for collaborative course development [SAM24].

LiaScript differs from all of the above in combining Markdown simplicity, rich interactive affordances, and a fully client-side execution model that requires neither a server nor a pre-installed runtime [BS25].

2.2 Feature Adoption in Educational Technologies

A recurring finding in educational technology research is that the number of available features in a system far exceeds the number of features that are actually used. Studies on learning management systems document this pattern consistently: Hijazi et al. [Hi20] found a strong positive correlation between feature *awareness* and feature *usage* in Moodle, with advanced features such as SCORM packages and workshop modules remaining largely unknown and therefore unused. Awad et al. [Aw19] observed similarly uneven utilisation of LMS features even at well-resourced institutions.

The Technology Acceptance Model (TAM) and its extensions provide a theoretical framework for these observations [Gr22]. Feng et al. [Fe25] identify *Effort Expectancy* — the perceived complexity of using a feature — as the dominant predictor of non-adoption in higher education contexts, outweighing perceived usefulness for features that require significant authoring effort.

Crucially, all of these studies were conducted on server-based systems that generate usage logs. No prior work analyses feature adoption in a decentralised, client-side OER tool where no telemetry is available and repository content is the only observable proxy for usage behaviour.

2.3 Mining Educational Repositories

Mining software repositories (MSR) is an established methodology for understanding how tools are used in practice [Co16; Co17]. Schröder and Pfänder [SP20] specifically explore GitHub as a platform for OER, and Santos-Hermosa et al. [Sa17] analyse reuse indicators across OER repositories. Both studies confirm that repository metadata — commit history, contributor counts, file content — provides meaningful signals about authoring behaviour, while also cautioning about the incompleteness of public repository data as a usage proxy.

Our work extends this line of research by applying corpus analysis to a specific OER authoring language, treating the GitHub corpus as the *only available substitute* for telemetry data in a privacy-preserving, client-side tool.

3 Dataset and Methodology

3.1 Data Collection Pipeline

The dataset was constructed through a nine-stage pipeline (Table 1), implemented in the open-source *ConnectedLecturer* project.[3] Starting from GitHub's Search API, we identified 1.076 repositories containing potential LiaScript content. A tree-level aggregation step extracted 57.096 Markdown files, of which 3.317 (5.8 %) were validated as LiaScript courses using a combination of rule-based filters and an LLM classifier (Llama 3.3 70B). For each course, the pipeline collected commit history (19.366 commits), parsed header metadata (99.4 % success rate), extracted 44 binary feature flags, and performed AI-based enrichment (titles, keywords, educational level) for 94.6 % of courses. The final consolidated dataset comprises 3.317 courses with 166 columns each (collected 2026-03-13).

Tab. 1: Data collection pipeline stages

Phase	**Method**	*n*
Repository search	GitHub Search API	1.076 repos
File extraction	Tree API + heuristics	57.096 files
Validation	Rules + LLM	3.317 courses (5.8 %)
Commit analysis	Commits API	19.366 commits
Header metadata	Regex parsing	3.297 courses (99.4 %)
Feature detection	44 regex patterns	3.317×44
AI enrichment	RAG + Llama 3.3 70B	3.138 courses (94.6 %)
User profiles	GitHub Users API	445 profiles
Consolidation	Inner join	3.317×166

Our primary unit of analysis is the individual *course file* (one validated `.md` file = one course). For grouping purposes courses are segmented by their *repository context* (Section 3.3).

3.2 Feature Taxonomy

For each course file the pipeline extracts 44 binary feature flags via syntactic pattern matching on the Markdown source.[4] We organise these features into four didactically motivated categories (Table 2).

Each category addresses a different pedagogical barrier. *Presentation* features target accessibility (screen-reader-friendly TTS, animated step-by-step reveals). *Interaction* features enable formative self-assessment without a learning management system. *Reuse* features

3 `https://github.com/TUBAF-IFI-ConnectedLecturer/Data_aggregation`

4 All regex patterns and their mapping to feature labels are documented at `https://github.com/TUBAF-IFI-ConnectedLecturer/Data_aggregation/blob/main/cl-pipeline/run/pipelines/liascript/docs/LiaScript_Features_Patterns.md`

Tab. 2: LiaScript feature taxonomy (4 categories, 44 tracked features)

Category	Design intent	Representative features
Presentation	Self-paced, accessible delivery	Narrator (TTS), animations, effects, galleries
Interaction	Active learning & formative feedback	Single/multiple choice, text input, matrix quiz, surveys
Reuse	Scalable authoring via shared components	Imports, custom macros, official templates
Embedding	Hands-on executable environments	Executable code, WebApps, external scripts, ASCII diagrams

support the Don't-Repeat-Yourself principle at course level, allowing shared building blocks across a curriculum. *Embedding* features close the gap between explanation and practice by running code, simulations, or visualisations directly in the browser.

3.3 Three-Group Segmentation

Each course resides in a GitHub repository owned by an individual or organisation account. The 3.317 courses were contributed by 307 unique committers working across 205 repository accounts (167 individual, 37 organisational, 1 unclassified). Organisation accounts act as hosting containers; the actual authors are identified through the Git commit history (`contributors_list`).

To prevent distortions from authors with fundamentally different relationships to LiaScript, we segment courses into three groups based on the *repository context*:

Internal (462 courses, 7 repository accounts, 58 committers): Repositories owned by the core development team and their organisational accounts (SebastianZug, andre-dietrich, LiaPlayground, LiaScript, LiaBooks, LiaTemplates, TUBAF-IfI-LiaScript). These include templates, demonstrations, and the developers' own teaching materials.

MINT-the-GAP (1 147 courses, 1 organisation account, 3 committers): A single prolific school-focused initiative producing structured STEM courses. With nearly a third of all courses, this account would dominate any community-level analysis if not separated.

Community (1 708 courses, 197 repository accounts, 269 committers): All remaining courses, representing independent adoption by educators without direct involvement in LiaScript development. The group spans a wide range of engagement: 82 accounts have produced exactly one course — including test repositories, workshop materials, and one-off teaching projects — while 8 accounts each contribute 56–146 courses. Despite this spread, no single account dominates (the largest accounts for 8.5 %), and feature adoption rates show no systematic divergence between prolific and occasional

contributors. Excluding all single-course accounts shifts adoption rates by less than one percentage point, confirming that the group is not distorted by test or tutorial content.

Because grouping follows the repository context, the same committer may appear in multiple groups. In particular, 21 committers — including the two core developers — contribute to both Internal and Community repositories. Their work in Community repositories is counted as Community, reflecting the collaborative context rather than the contributor's identity. For 306 courses (9.2 %) no commit history was available; these are assigned solely based on their repository account.

3.4 Complementary User Survey

To triangulate the corpus findings, we conducted a structured needs assessment with seven active LiaScript authors from different institutions (March 2026). Participants prioritised feature requests and usability issues on a five-point scale. The resulting 60 items are analysed qualitatively in Section 5.

3.5 Limitations

The corpus is restricted to *public* GitHub repositories. Courses hosted on other platforms, behind authentication, or in private repositories are not captured. Feature detection via pattern matching may miss features used through indirect mechanisms (e.g. macros that generate quizzes programmatically). The three-group segmentation assigns courses by repository context, so the 21 committers active in multiple groups are counted in each group separately rather than as a single entity. The AI enrichment step (titles, keywords, DDC classification) was not used in the present analysis but is available for future work.

4 Feature Adoption in the Wild

Across the full corpus of 3.317 courses, feature adoption varies widely both between and within the four taxonomy categories. Table 3 lists all tracked features sorted by adoption rate within each category, revealing a clear gradient from broadly adopted to virtually unused features.

4.1 Aggregate Adoption Rates

The table reveals several patterns. Presentation features split sharply: narrator and animations require minimal authoring effort (a single comment line activates TTS) and achieve broad

Tab. 3: Adoption rates across all 3.317 courses, sorted by rate within each category. Features above 10 % are set in the left column, those below in the right.

	Widely adopted		Rarely adopted	
Category	**Feature**	**%**	**Feature**	**%**
Presentation	Narrator	58.6	Galleries	7.3
	Animations	22.8	Animated CSS	0.7
	TTS fragments	11.6	TTS blocks	0.1
	Anim. blocks	11.0	Effects	0.0
Interaction	Any quiz	45.5	Single choice	9.0
	Text quiz	28.8	Quiz hints	4.9
	Multiple choice	10.2	Task lists	4.7
			Surveys	2.7
			Matrix quiz	1.6
Reuse	Macros (any)	72.1	Custom macros	8.7
	Imports	59.6		
	Ext. scripts	51.1		
Embedding	Math	48.0	ASCII diagrams	9.8
	Script tags	20.8	WebApps	5.6
	HTML embeds	15.7	Exec. code	3.1
			Code projects	0.4

uptake, while fine-grained variants remain near zero. Reuse features are driven by the import and macro system — most authors *consume* shared libraries and macros but few *define* custom macros (8.7 %). Interaction is dominated by quizzes (45.5 %), but survey and matrix quiz types remain rare. Embedding features show the widest internal spread: math notation reaches 48.0 % in STEM-oriented courses, while executable code (3.1 %) and code projects (0.4 %) remain niche despite being a signature LiaScript capability.

4.2 Group-level Adoption Profiles

Table 4 lists the features with notable adoption differences across the three groups (spread > 10 percentage points). Features with similar adoption rates across groups are omitted from the table and discussed below.

Three patterns emerge from these divergences:

MINT-the-GAP's standardised configuration. MINT-the-GAP dominates the Reuse category (macros 99.7 %, imports 98.3 %, external scripts 96.3 %) and relies heavily on text quizzes (70.0 %) and math notation (95.2 %). This profile reflects a standardised configuration for small, self-contained STEM exercises: each course file imports a common

Tab. 4: Features with divergent adoption rates across groups (spread > 10 pp). The highest value per row is set in **bold**.

Category	Feature	Internal	MINT	Community
Presentation	Narrator	**86.6**	9.4	84.0
	TTS fragments	**36.1**	0.1	12.6
	Animations	**66.2**	0.6	25.9
	Anim. blocks	**36.8**	0.3	11.3
	Galleries	**17.3**	0.0	9.5
Interaction	Any quiz	22.7	**78.3**	29.6
	Single choice	11.9	0.2	**14.1**
	Multiple choice	11.3	0.8	**16.3**
	Text quiz	9.7	**70.0**	6.2
Reuse	Macros (any)	69.0	**99.7**	54.4
	Imports	57.4	**98.3**	34.3
	Ext. scripts	16.2	**96.3**	30.3
Embedding	Exec. code	**11.7**	0.3	2.7
	WebApps	**13.0**	0.7	6.9
	Script tags	**33.3**	8.2	25.8
	ASCII diagrams	**38.5**	2.3	7.1
	Math	30.7	**95.2**	21.0
	HTML embeds	18.6	5.9	**21.5**

set of libraries (TikZ visualisation, Algebrite CAS) and uses text quizzes as the primary interactive element. Notably, presentation features like narrator (9.4 %) and animations (0.6 %) are nearly absent — the exercise format does not call for them.

Internal as feature showcase. The Internal group leads in presentation features (narrator 86.6 %, animations 66.2 %) and embedding features (ASCII diagrams 38.5 %, script tags 33.3 %). This is consistent with the developers' role as feature demonstrators and early adopters of the full language surface.

Community as selective adopters. The Community group generally falls between the other two, but with notable exceptions: single-choice (14.1 %) and multiple-choice quizzes (16.3 %) are *higher* than in Internal, suggesting that community authors prefer familiar quiz types over text-input quizzes. HTML embeds (21.5 %) also peak in the Community group.

Convergent features. Several features show similar adoption across all three groups: matrix quizzes (0.3 % bis 2.4 %), surveys (0.0 % bis 4.8 %), task lists (0.0 % bis 8.7 %), and code projects (0.0 % bis 1.3 %). These consistently low rates indicate features that remain

underutilised regardless of author background, pointing to adoption barriers that transcend group-specific workflows.

5 Development Implications

The adoption data raises a fundamental strategic question: should development effort target the many occasional authors who plateau at narrator and quizzes, or the few prolific authors who might exploit the full feature space? The corpus provides a clear empirical answer. Of the 197 Community accounts, 82 have produced exactly one course and 81 have produced two to ten. Yet the eight most prolific accounts (56–146 courses each) exhibit essentially the same feature profile as occasional contributors (Section 3.3). Production volume does not drive feature exploration. This means that investing exclusively in power-user features (e.g. code projects, adaptive learning paths) addresses a small audience without broadening the overall feature uptake. Conversely, the MINT-the-GAP model demonstrates that *infrastructure-level* choices — a standardised configuration that pre-selects shared libraries and exercise formats — can lift adoption of specific features for an entire author community at once. The implication is that the most effective improvements operate at the boundary between casual and intermediate use, lowering the step from the current adoption plateau to the next rung of the feature ladder.

The adoption gradient itself can be explained through three gap classes.

5.1 Three Adoption Gap Classes

Documentation deficit. Some features are syntactically non-obvious and lack progressive examples. Consider the contrast between narrator and effects: activating text-to-speech requires a single header line (`<!-- narrator: German Female -->`), whereas timed content appearance demands a numeric fragment notation (`{{1}}`, `{{2-3}}`) combined with block delimiters (`--{{1}}--`) for accompanying audio — a secondary syntax layer documented primarily through a single, comprehensive reference rather than graduated tutorials. The strong adoption of narrator (84 % in the Community) and basic macros (54 %) shows that authors *do* progress beyond plain Markdown when the entry cost is low. The three-group comparison reinforces this interpretation: Community authors show significantly lower adoption of animation and script features than Internal authors, although the underlying syntax is identical — suggesting that existing documentation does not sufficiently bridge the gap. A survey participant independently confirmed this barrier: *"Kompaktere, auszugsweise Dokumentation für Einsteiger"* (priority 4/5), while another requested *"Einführung verschiedener Rollen [im Editor], die unterschiedliche Anzahl von Feature-Buttons gezeigt bekommen (Novice, Power User)"* (priority 4/5).

Workflow barrier. Code projects and executable code require authors to configure external services (CodeRunner templates via `import:`) and understand the interaction between named code blocks (+/- visibility prefixes, filename annotations). For educators without a technical background this represents a barrier that documentation alone cannot resolve. The survey confirms this concretely: *"Bildausgaben beim Coderunner sind umständlich, das Terminalfenster muss erst vergrößert werden"* (priority 3/5), and *"Devserver produziert anderen Code als die Live Version"* (priority 5/5), indicating that even experienced authors encounter friction in the embedding workflow. Priorities include a visual *template picker* that inserts pre-configured embedding blocks, improved error messages when external templates fail, and a default code execution sandbox requiring no configuration.

Configuration lock-in. The MINT-the-GAP profile demonstrates that a standardised configuration — importing the same libraries (TikZ, Algebrite) across all exercise files — can drive near-universal adoption of specific features (math 95 %, imports 98 %), but simultaneously leaves features outside this scope unexplored (narrator 9 %, animations 0.6 %). This pattern represents a design tension: standardised configurations lower the entry barrier for a predefined feature set but discourage exploration beyond it. The survey reveals related friction: *"Kompatibilität zu Standard-MD herstellen [. . .] Es gibt eine größere Liste von Situationen wo sich LiaScript-MD nicht wie echtes MD verhält"* (priority 4/5) points to frustration when LiaScript-specific syntax diverges from author expectations. Future shared configurations should expose optional extension points so that standardised workflows do not inadvertently narrow the feature space.

Convergent adoption floors. Several features remain below 5 % in *all* three groups, including among the core developers themselves (animated CSS 2.4 %, code projects 1.3 %, effects 0 %). Unlike the documentation deficit, where Internal adoption proves that the feature *can* succeed, these convergent floors suggest that the features' current design may not match real authoring needs. They warrant a critical review of whether the feature concepts themselves require rethinking rather than better documentation.

5.2 The Adoption Ladder

Community adoption rates suggest a natural progression path: narrator and links (> 80 %) → macros and tables (≈ 50 %) → quizzes and animations (≈ 25 %) → embedding features (< 10 %). The power-user analysis in Section 3.3 shows that prolific Community authors (8 accounts with 56–146 courses each) exhibit essentially the same feature profile as occasional contributors, indicating that production volume alone does not drive feature exploration. Onboarding documentation should therefore follow this ladder explicitly, guiding authors from high-adoption to low-adoption features in a structured sequence rather than presenting all capabilities as equally accessible.

5.3 Qualitative Triangulation

Beyond the individual quotes cited above, the survey reveals a cross-cutting theme: four of the 60 items request *AI-assisted course generation*, making it the most prominent cluster. One participant described an *"Agentensystem zur Kurserstellung mit Human-in-the-middle [. . .] Die AI stellt Rückfragen und erstellt dann einen Kurs, dieser wird von einem AI-Linter gegengecheckt"* (priority 2/5). Combined with the configuration lock-in finding, this suggests that AI-based generation could fulfil the same adoption-enabling role as standardised configurations — lowering the entry cost for complex features — while avoiding the narrowing effect of fixed setups. Two further items request a *syntax checker or linter* (*"LiaScript Linter"*, priority 2/5; *"Bessere Fehlermeldungen bei Syntaxfehlern"*, priority 3/5), reinforcing that real-time authoring feedback is perceived as insufficient across experience levels.

5.4 Monitoring Future Adoption

Because LiaScript's architecture precludes runtime telemetry, repeating the GitHub corpus analysis at regular intervals is the only mechanism available for tracking whether these improvements translate into changed authoring behaviour. We recommend treating the corpus snapshot as an annual instrument, analogous to a developer survey in the open-source context.

6 Conclusion

This paper presented the first systematic empirical analysis of feature adoption in the LiaScript ecosystem. Exploiting the GitHub corpus as the only observable proxy for a tool with no server-side telemetry, we analysed 3.317 courses from 307 committers using a four-category feature taxonomy.

The central finding is a pronounced adoption gradient. Presentation features — particularly narrator/TTS and structural animations — are widely used, as is the macro and import system for reuse. Interactive affordances show selective adoption, while advanced embedding features remain nearly invisible in practice despite being core components of the LiaScript design.

This gradient mirrors a well-documented pattern in software engineering: industry reports consistently find that roughly 80 % of features in software products are rarely or never used [Pe19]. Our three-group comparison adds nuance to this observation by showing that different author communities exploit *different* feature subsets — MINT-the-GAP's standardised configuration achieves near-universal adoption of reuse and math features but leaves presentation capabilities unexplored, while Community authors follow a natural

adoption ladder that plateaus well below the full feature space. The HCI principle of *progressive disclosure* [Ni06] — revealing complexity only as users need it — maps directly onto this ladder and should guide future documentation design. Notably, prolific authors show the same feature profile as occasional contributors, indicating that experience alone does not drive exploration — a finding that underscores the need for *active* onboarding rather than relying on organic discovery.

These findings translate into three actionable priorities: progressive documentation following the adoption ladder, reduced embedding complexity through better tooling, and shared configurations that expose rather than hide optional features. AI-assisted course generation, identified as the top priority in a complementary user survey, may address all three barriers simultaneously.

A companion study will shift the lens from feature adoption to course *lifecycle*: how courses evolve over time, how collaboration manifests through commits and forks, and whether GitHub engagement metrics (stars, forks) serve as valid indicators of educational impact in a system that, by design, leaves no usage trace.

Literaturverzeichnis

[21] You Only Write Thrice: Creating Documents, Computational Notebooks and Presentations From a Single Source. In: ArXiv. arXiv:2107.06639, 2021.

[Aw19] Awad, M. et al.: Evaluating Learning Management System Usage at a Small University. In: Proceedings of the 2019 3rd International Conference on Information System and Data Mining. 2019, DOI: 10.1145/3325917.3325929.

[Be25] Betlem, P. et al.: Jupyter Book as an open online teaching environment in the geosciences: lessons learned from Geo-SfM and Geo-UAV. Geoscience Communication, 2025, DOI: 10.5194/gc-8-51-2025.

[BS25] Baur, M.; Speiser, S.: A Practical Evaluation of LiaScript as a Collaborative Teaching Technology. In: Proceedings of the 11. Fachtagung Hochschuldidaktik Informatik (HDI 2025). ISBN 978-3-00-083968-9, Freiberg, Saxony, S. 179–187, 2025, DOI: 10.5281/zenodo.16893818.

[Co16] Cosentino, V. et al.: Findings from GitHub: Methods, Datasets and Limitations. In: 2016 IEEE/ACM 13th Working Conference on Mining Software Repositories (MSR). S. 137–141, 2016, DOI: 10.1145/2901739.2901776.

[Co17] Cosentino, V. et al.: A Systematic Mapping Study of Software Development With GitHub. IEEE Access 5, S. 7173–7192, 2017, DOI: 10.1109/ACCESS.2017.2682323.

[Di19] Dietrich, A.: LiaScript: A Domain-Specific-Language for Interactive Online Courses. In: Proceedings of the International Conference on e-Learning. S. 186–194, 2019.

[Fe25] Feng, J. et al.: Key factors influencing educational technology adoption in higher education: A systematic review. PLOS Digital Health 4, 2025, DOI: 10.1371/journal.pdig.0000764.

[Fo25] Fornasari, A. et al.: Authoring tools for e-learning: critical analysis and pedagogical perspectives. The case of H5P in interactive digital teaching. ELEARNING2025, 2025, DOI: 10.46793/elearning2025.077f.

[Gr22] Granić, A.: Educational Technology Adoption: A systematic review. Education and Information Technologies 27, S. 9725–9744, 2022, DOI: 10.1007/s10639-022-10951-7.

[Hi20] Hijazi, H. et al.: Investigating the Awareness and Usage of Moodle Features at Hashemite University. International Journal of Advanced Computer Science and Applications 11, 2020, DOI: 10.14569/ijacsa.2020.0110625.

[HO18] Henderson, S.; Ostashewski, N.: Barriers, incentives, and benefits of the open educational resources (OER) movement: An exploration into instructor perspectives. First Monday 23, 2018, DOI: 10.5210/FM.V23I12.9172.

[Ju25] Jupyter Project et al.: Jupyter Book 2 and the MyST Document Stack. 2025, DOI: 10.25080/hwcj9957.

[Ki22] Kiryakova, G.: Engaging Learning Content for Digital Learners. TEM Journal, 2022, DOI: 10.18421/tem114-65.

[KW17] Krewinkel, A.; Winkler, R.: Formatting Open Science: agilely creating multiple document formats for academic manuscripts with Pandoc Scholar. PeerJ Preprints 5, e2648, 2017, DOI: 10.7287/peerj.preprints.2648v2.

[Mo25] Moser, M.; Hamann, T.; Abdelrazeq, A.; Schmitt, R. H.: Leveraging GitLab as a Platform for the Provisioning and Managing of Open Educational Resources in an Open Educational Practices Use Case. Bibliothek Forschung und Praxis 49 (3), S. 365–378, 2025, DOI: 10.1515/bfp-2025-0026.

[Ni06] Nielsen, J.: Progressive Disclosure. Nielsen Norman Group, 2006, URL: https://www.nngroup.com/articles/progressive-disclosure/.

[Pe19] Pendo.io: The 2019 Feature Adoption Report, Industry report on SaaS feature usage patterns, 2019, URL: https://www.pendo.io/resources/the-2019-feature-adoption-report/.

[Sa17] Santos-Hermosa, G. et al.: Repositories of Open Educational Resources: An Assessment of Reuse and Educational Aspects. The International Review of Research in Open and Distributed Learning 18, S. 84–120, 2017, DOI: 10.19173/IRRODL.V18I5.3063.

[SAM24] Schwiertz, G.; Arnold, D.; Mühlichen, A.: Offene Formate für offene Lehrmaterialien. In: 112. BiblioCon Hamburg 2024 (Hands-on Lab). Demonstrates Markdown + LiaScript + GitHub workflow for OER, 2024.

[SP20] Schröder, N.; Pfänder, P.: Nutzung von GitHub für Open Educational Resources, Semantic Scholar: 98970e1434c3e112023aaac2d7f174f02cd0f268, 2020.